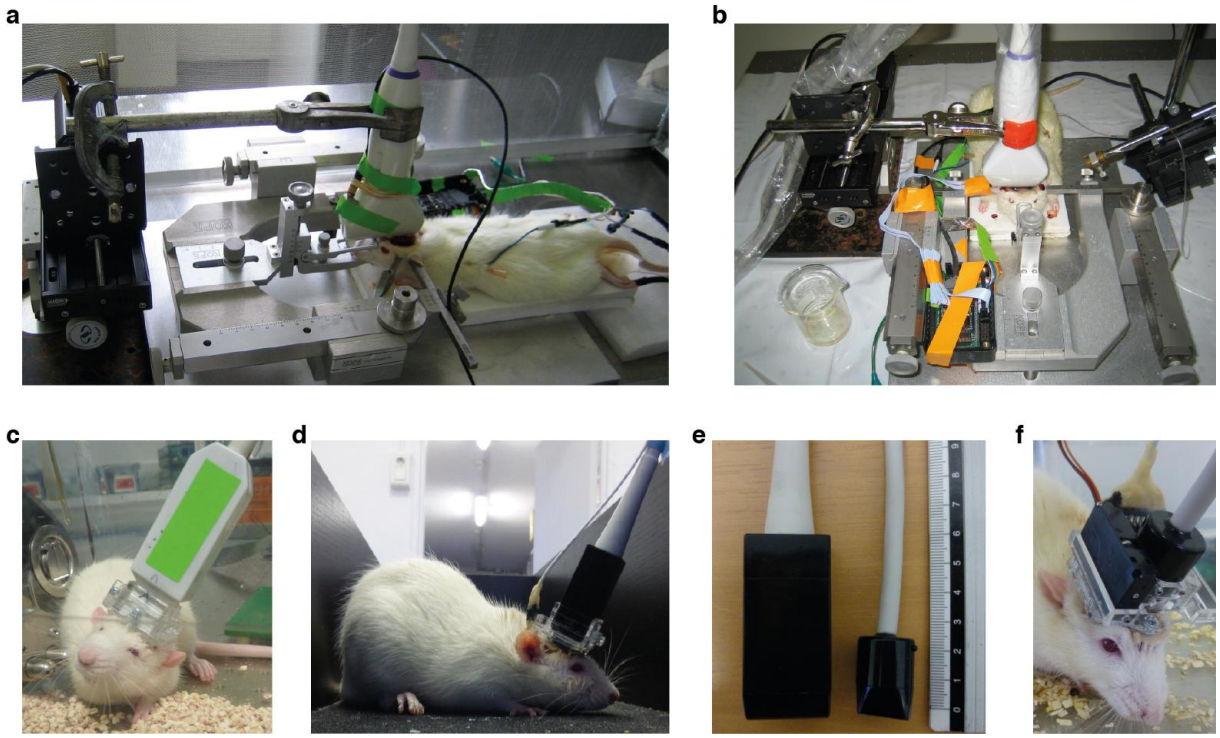


Supplementary Figure 1

Comparison of static and awake mobile fUS

The mfUS method is based on incremental improvements over functional ultrasound imaging on anesthetized rats.

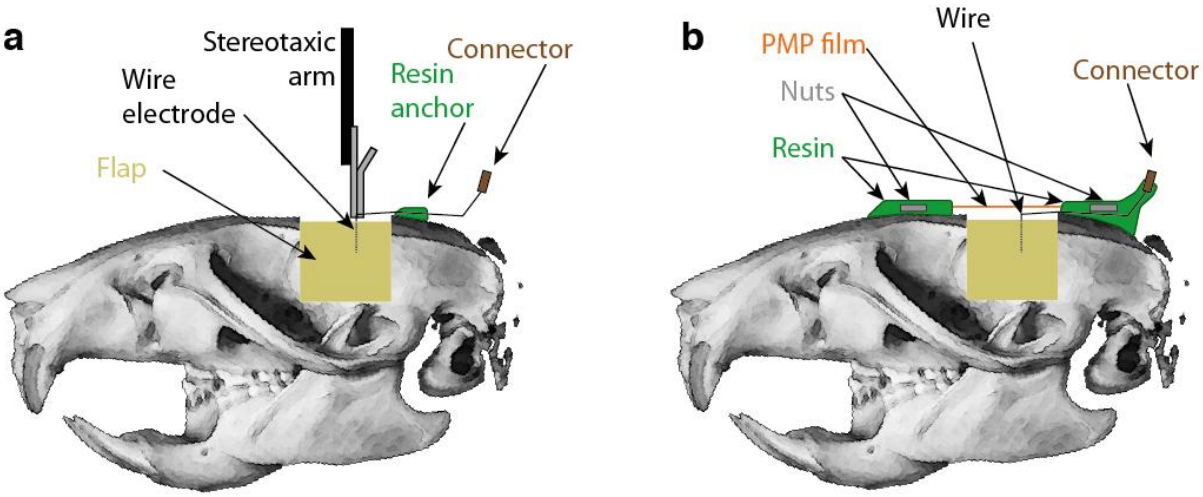
(a, b) show side and front views of static fUS on anesthetized rat with open or thinned-bone craniotomy [5, 7]. The rat is sedated and fixed in a stereotaxic frame.

(c) the PMP prosthesis surgical procedure and a first downsized ultrasound probe provide images in awake condition although the animal is highly constrained by the weight of the probe.

(d) a second generation, smaller and lighter, probe allows the animal to move and EEG to be recorded.

(e) comparison of second and third generation of probe show a further gain in size and weight.

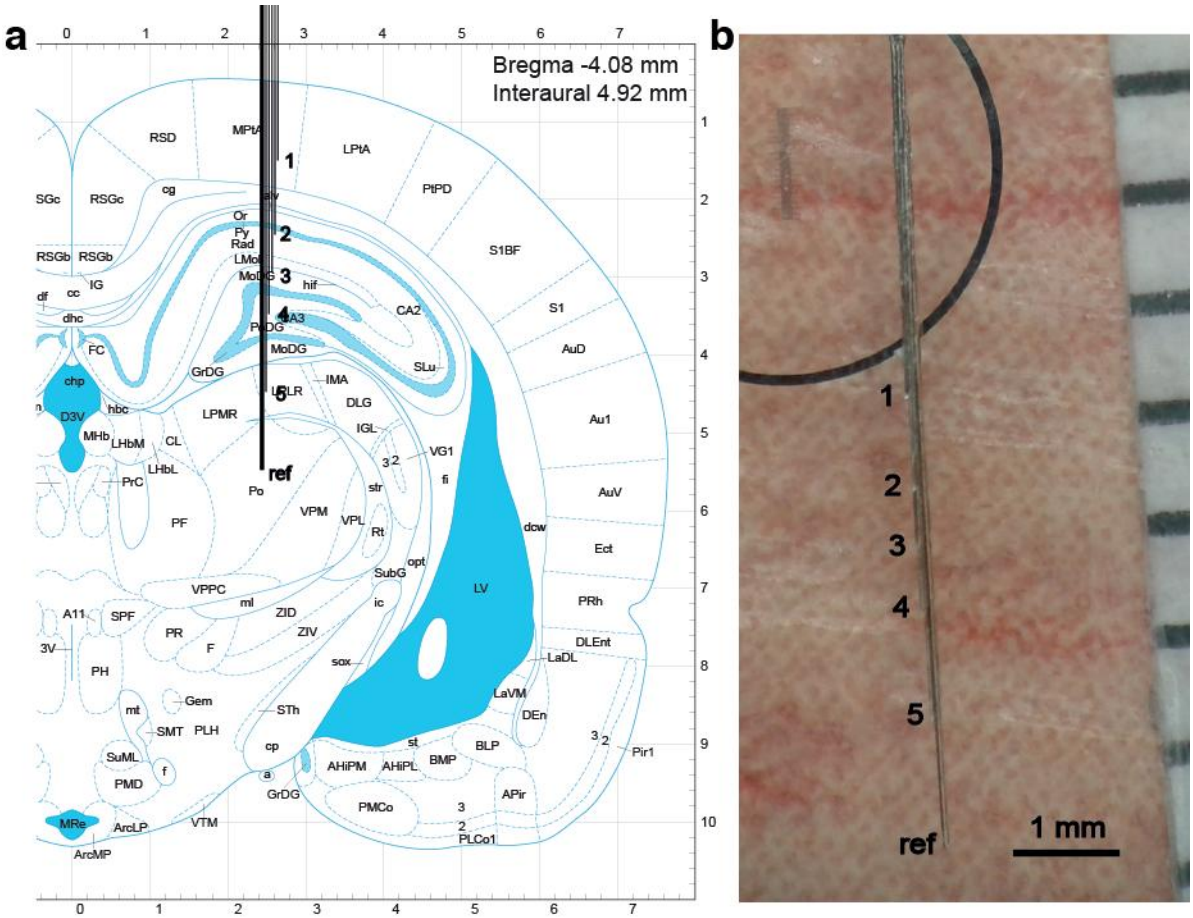
(f) miniature motorized design allows to scan multiple planes without directly interacting with the animal, while the mechanical constraint is minimal. All the navigation and epilepsy measures presented here were performed with this smaller design.



Supplementary Figure 2

Surgical procedure

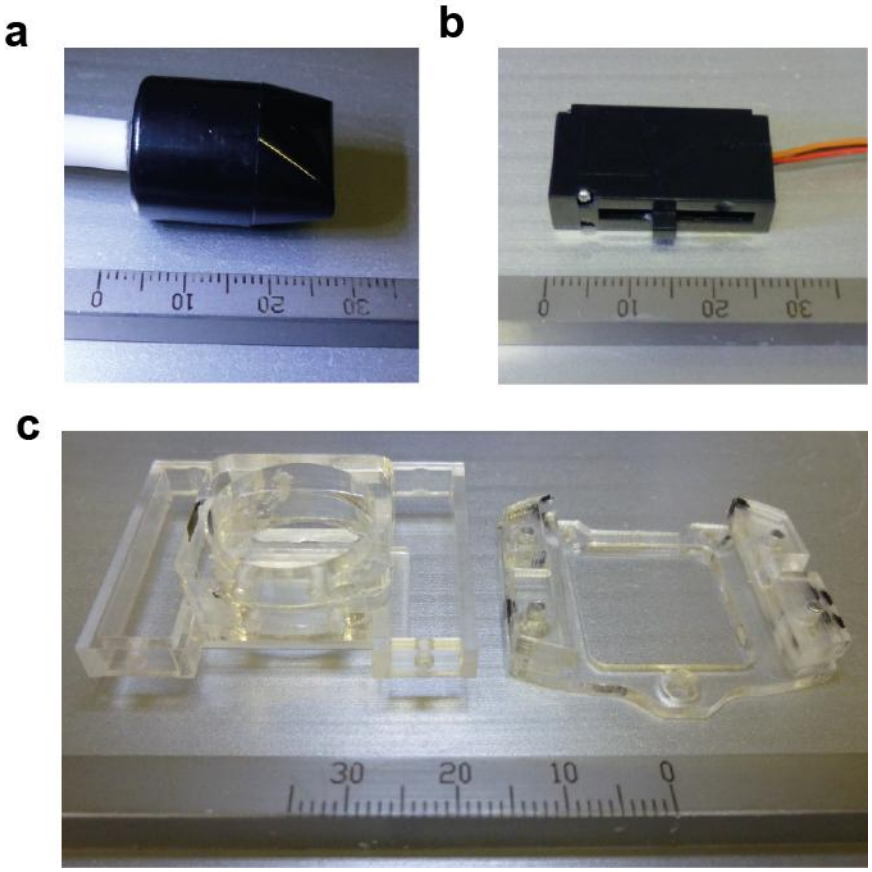
The surgical procedure to prepare the rat for simultaneous EEG and fUS recording proceeds in successive steps. First, a large bone flap is removed. Then electrodes are moved to the target position by stereotaxic micromotion, and they are anchored to the edge of the flap with a droplet of resin (**a**). For clarity, only one electrode is shown. Following sequential insertion of all the electrodes, the skull is replaced by an ultrasound clear prosthesis made of PMP. Finally, the prosthesis is sealed to the skull with resin and nuts are embedded in the resin for later attachment of the probe holder (**b**).



Supplementary Figure 3

Hippocampal electrodes

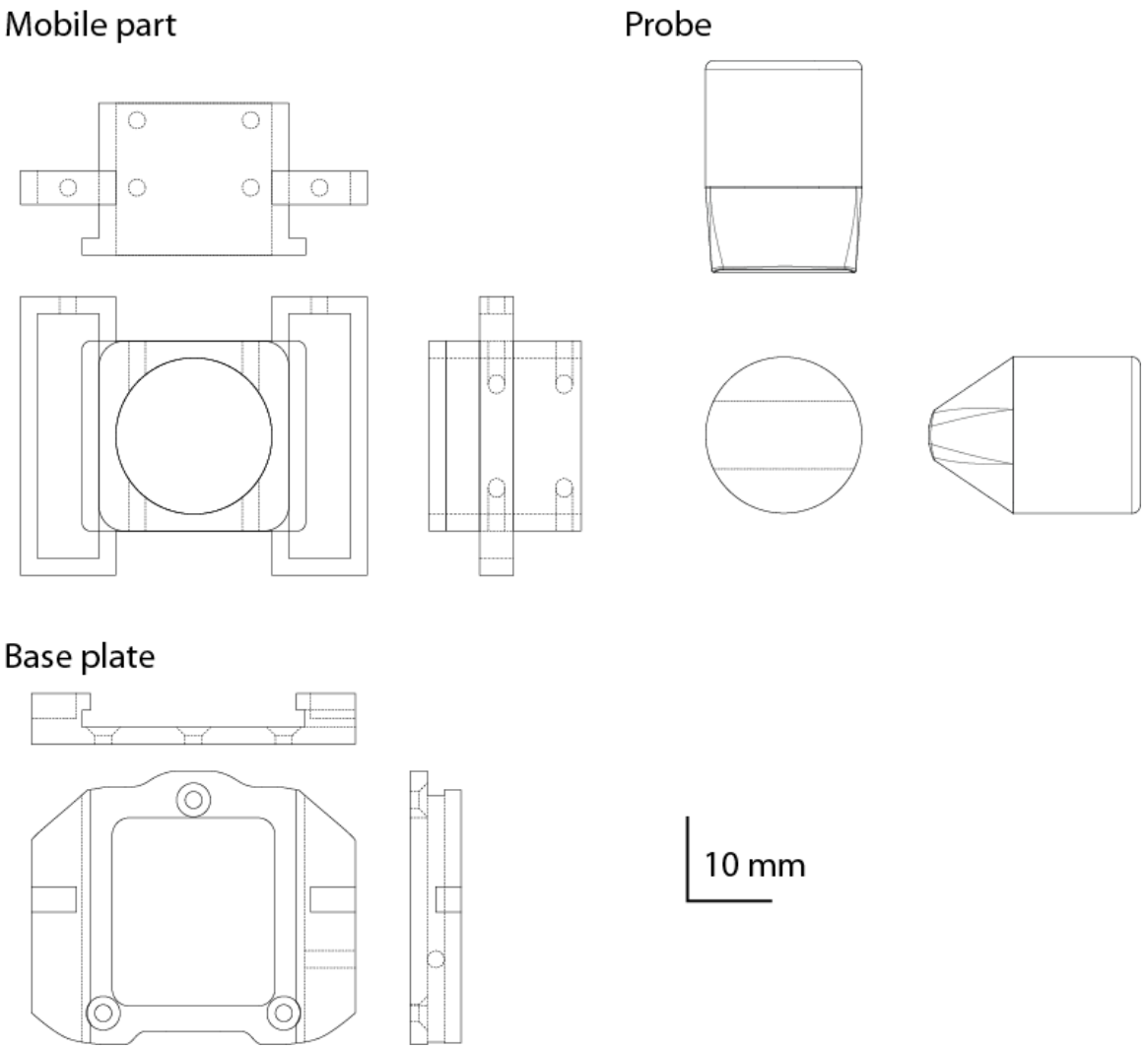
Hippocampal electrodes are made from a bundle of insulated tungsten wires soldered to a miniature connector at one end. Under binocular control the 50 µm reference electrode is juxtaposed with five 25 µm recording electrodes. The wires tips are positioned at fixed spacing in order to reach anatomical structures encompassing the dorsal hippocampus (**a**) and they are glued with cyanoacrylate in the upper part of the bundle to avoid accidentally covering the recording tips. A millimeter scale and microscope scale slide (total length 1 mm) are used to precisely set the spacing. Atlas from Paxinos[19] (**b**). Differential recording from electrode tips 2-4, that show the typical theta band activation in relation with spatial navigation and phase reversal along the dorso-ventral axis are retained for analysis.



Supplementary Figure 4

Probe, holder and motor

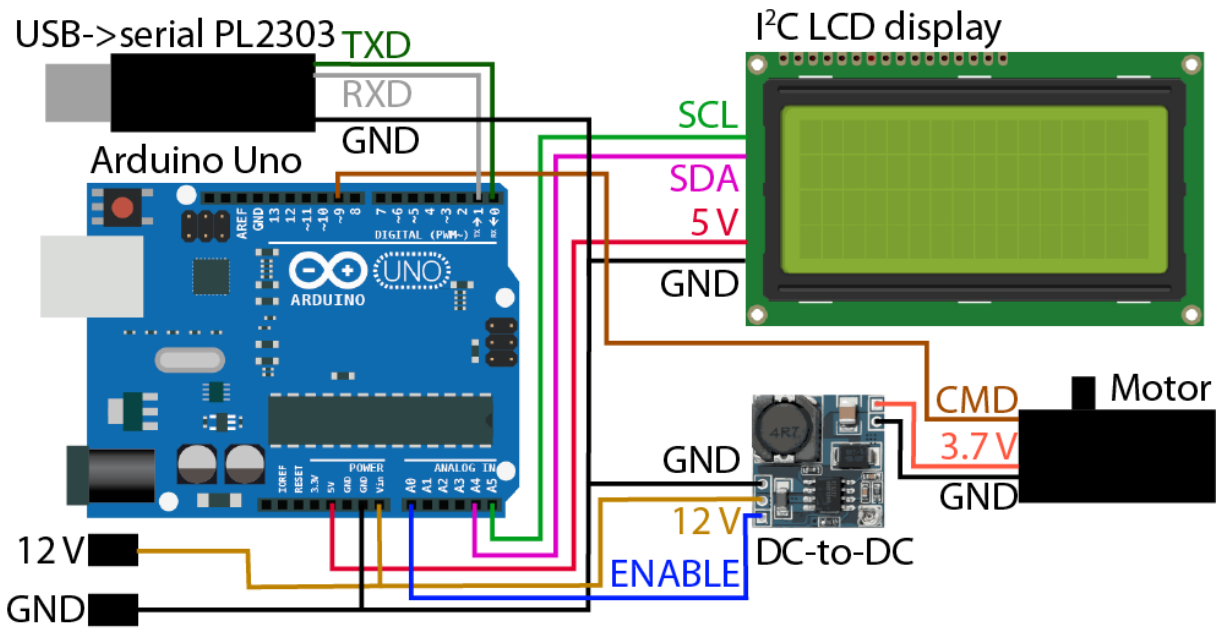
Photographs of the miniature ultrasound probe (**a**), micromotor (**b**). The probe holder (**c**) is composed of a plate fixed onto the head (right) and a translation stage (left).



Supplementary Figure 5

CAD drawing of probe and holder parts

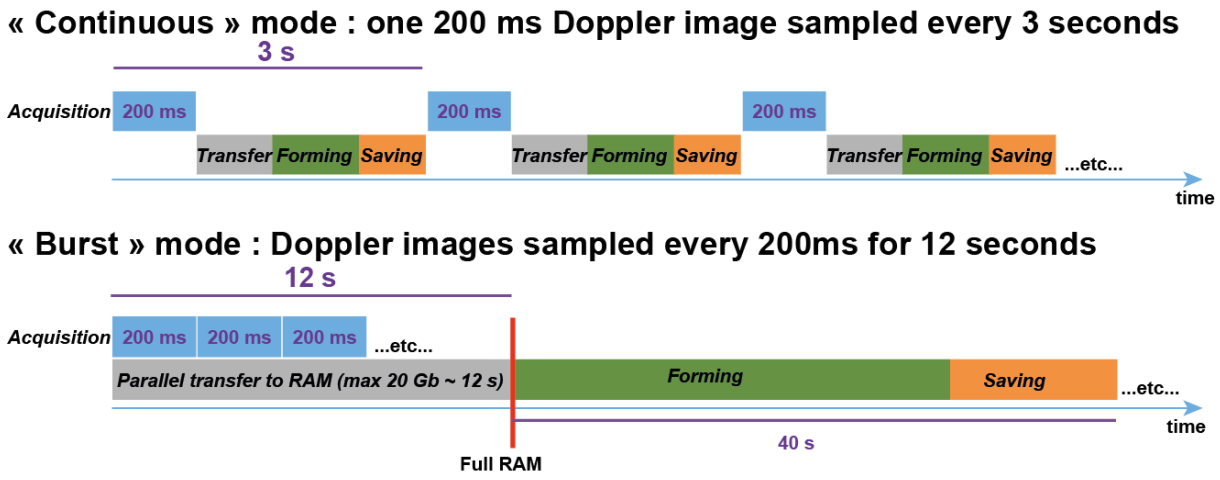
Front, side and top views of the base plate fixed to the head, mobile holder part and probe that fits into the mobile part. The base plate is attached to the head with M2 conical head screws. The probe and motor are secured with M2 headless screws. The sliding surface is lubricated with teflon spray. Digital information to reproduce the parts is provided as **Supplementary Material 1**.



Supplementary Figure 6

Motor controller circuit

The motor controller circuit is based on an Arduino Uno open-source electronic platform. Requests for mechanical translation are received from the ultrasound scanner through a USB to serial converter based on a PL2303 chip. The Arduino turns on a DC-to-DC step down converter through the "ENABLE" pin to power the servomotor with optimal 3.7V voltage. Simultaneously the position information is sent by PWM modulation to the command line. Motor power is turned off. An LCD provides status information. Arduino microcode is provided in **Supplementary Note**.



Supplementary Figure 7

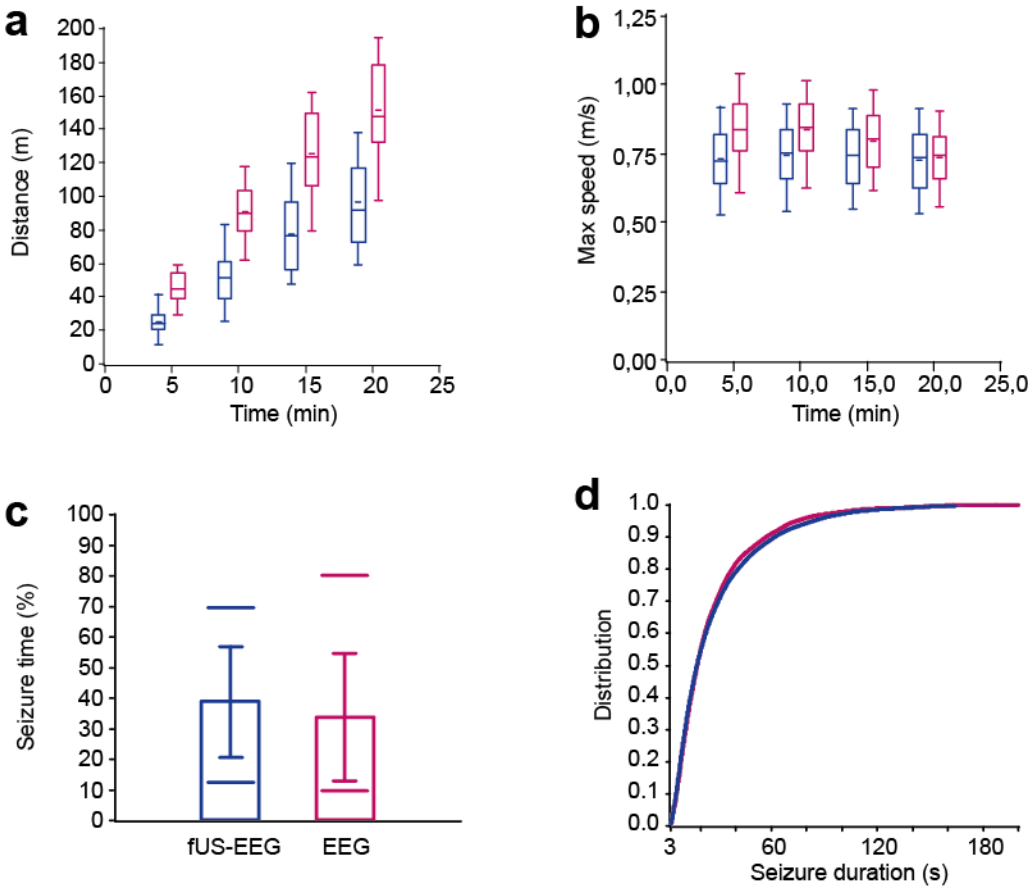
Comparison of "continuous" and "burst" modes

The continuous and burst modes make distinct usage of the ultrasound scanner acquisition and processing power. The main task is to transfer and process the large amount of high frequency plane wave scans into an fUltrasound images, which is saved as an array of single precision encoded pixel Power Doppler intensity corresponding to CBV. The reduction in data size is 1000 fold since 200 scans composed of repeats at 5 angles are used to generate one fUltrasound image.

The "continuous" mode has the advantage to permit acquisition of virtually unlimited duration. For this mode, the Ultrafast acquisition is set to form a single fUS image (200 ms). The transfer and processing time (here 2 seconds) limits the final rate of CBV maps.

The "burst" mode has the advantage to sustain the Ultrafast frame rate up to the filling of the RAM memories (here 12 s), allowing to capture short behavioral events such as maze crossing. CBV maps are reconstructed every 200 ms, but for this limited 12 s duration. Data is transferred in parallel with acquisition, and when buffer memory is full data is processed (40 s) to form a series of fUltrasound images. Thus duration of acquisition is limited by memory size, and interval duration is limited by processing speed.

The use of two modes is only due to technical limitations of current electronic boards of the ultrasound system. With improving bus technology and high end GPU boards it should be possible in theory today to build an Ultrafast ultrasound scanner with transfer and processing rates fast enough to work in the continuous mode during unlimited amount of time.



Supplementary Figure 8

Effect of recording procedure on running and seizures

(a, b) Maze experiment comparative distributions between mfUS-EEG (blue) and control untethered, surgery free, rats (red) show cumulative distance travelled in the maze is reduced by 36-44% while maximum speed is moderately reduced by 1-13%. Graphs show mean, median, center quartiles, 1 and 99 percentiles. (**c, d)** Genetic absence rats (GAERS, blue) show equivalent seizing time fraction and seizure duration distribution compared to EEG-only recordings (red). Graph c shows mean+/- s.d., 1 and 99 percentiles.

Detailed statistics are given in the **Supplementary tables**.

Supplementary Tables

Table 1: Maze travelled distance (m).

	5min	10min	15min	20min
mfUS-EEG	25.1+/-8.4 n=19	51.2+/-17.1 n=19	77.2+/-23.4 n=18	96.0+/-26.8 n=18
Untethered	44.7+/-9.2 n=21	90.8+/-17.3 n=21	125.1+/-26.1 n=21	151.3+/-31.5 n=21
p (Student)	2E-8	1E-8	7E-7	1E-6
ratio	56%	56%	62%	64%

Cumulative distance travelled over time, from the beginning of the task, for EEG-mfUS and surgery-free, untethered, control rats. The two groups performed significantly differently, with a ratio of performance around 60%. n is the number of animal, Student t-test.

Table 2: Maze maximum speed (m/s)

	0-5min	5-10min	10-15min	15-20min
mfUS-EEG	0.728+/-0.120 n=191	0.739+/-0.131 n=202	0.737+/-0.119 n=181	0.726+/-0.124 n=135
Untethered	0.835+/-0.132 n=430	0.834+/-0.122 n=448	0.789+/-0.121 n=326	0.732+/-0.106 n=245
p (Student)	<1E-8	<1E-8	3E-6	0.59 (NS)
ratio	87%	89%	93%	99%

Maximum speed during maze end-to-end walk for successive 5min time windows, for EEG-mfUS and surgery-free, untethered, control rats. The two groups performed slightly, yet significantly, differently for the first 15 min, with a ratio of performance in the range of 87%-93. After 15min the difference between the two groups fell to 1%, which is not significant. n is the number of maze crossing, Student t-test.

Table 3: Absence seizure pattern

	Recording seizure fraction (%)	Seizure duration (s)
mfUS-EEG	38.8+/-18.2 n=65	26.5+/-30.2 n=6372
EEG	33.7+/-20.1 n=18	26.4+/-34.3 n=875
p	0.31, NS (Student)	0.98, NS (Kolmogorov)

Comparison of EEG seizure patterns in epileptic rats, between craniotomized EEG-mfUS rats and controls without cranial window (EEG group). The recording seizure fraction counts the proportion of time exhibiting seizure over the whole recording session. Duration of all detected seizures is also compared. Both indicators of seizure pattern showed non significant differences between the two groups, supporting a minimal effect of the experimental procedure on seizure activity. For fraction of time in seizure, n is the number of recordings. For seizure duration, n is the number of seizures. Tests are respectively Student and Kolmogorov.

Supplementary Note

Holder motor controller microcode

```
// Servo motor control from serial connection
// Ivan Cohen, 2015, INSERM U1130, Paris
/*
Circuit wiring:

 Arduino Uno

 Display:
 SDA pin A4
 SCL pin A5

 USB-serial adapter:
 GND
 RX1 pin 19
 TX1 pin 18

 Servo:
 PWM->pin9

 3.7V regulator:
 ENABLE->pin A0
*/
#include <stdlib.h> // for the atol function
#include <Servo.h> // servo control
#include <Wire.h> // I2C bus for display
#include <LiquidCrystal_I2C.h> // LCD display

LiquidCrystal_I2C lcd(0x27,20,4);  // init LCD for a 20 chars and 4 lines

// define variables
Servo myservo;
char cmdbuffer[8]="       ";
int bufpos=0;
int thisChar=0;
int value=0;
char displaybuffer[20];
int i,valid;
int OnOffPin=A0;
int r;

// init Arduino
void setup() {
// Open serial communications and wait for port to open:
  Serial1.begin(9600);
  Serial.begin(9600);
// init Servo
  myservo.attach(9,900,2100);  // attaches the servo on pin 9 to the servo object
// init display
  lcd.init();
  lcd.noBacklight();
  lcd.clear();
  lcd.backlight();
  lcd.print("Starting up...");delay(1000);
// init servo power pin
  pinMode(OnOffPin, OUTPUT);
  analogWrite(OnOffPin, 255);
  delay(1000);
  analogWrite(OnOffPin, 0);
// clear display
  lcd.clear();
  lcd.cursor();
  lcd.print("  Servo controller");
}

// Arduino main loop
void loop() {
// get serial input
  for(int i=0;i<8;i++) cmdbuffer[i]=0;
  bufpos=0;
  thisChar=0;
  lcd.setCursor(0,3);
  lcd.print("                ");
```

```
  lcd.setCursor(0,3);
  while((bufpos<4)&&(thisChar!=103))
    if (Serial1.available() > 0) {
       thisChar = Serial1.read();
       if(thisChar>0) {      cmdbuffer[bufpos]=thisChar;
       lcd.print(cmdbuffer+bufpos);
       bufpos++; Serial.println(thisChar);}
}

// interpret command
if((bufpos>0)&&(thisChar==103)) {
  valid=1;
  for(i=0;i<(bufpos-1);i++) valid=valid&&(cmdbuffer[i]>=48)&&(cmdbuffer[i]<=57);
  } else valid=0;

// execute valid command
if(valid) {
// convert serial input to integer
  value=atoi(cmdbuffer);
// display command value
  lcd.setCursor(0,3);
  sprintf(displaybuffer, "goto %4d", value);
  lcd.print(displaybuffer);
// turn on servo
  analogWrite(OnOffPin, 255);delay(500);
// move servo
  myservo.write(value); delay(2500);
// turn off servo
  analogWrite(OnOffPin, 0); delay(500);
// show current position
  lcd.setCursor(0,2);
  sprintf(displaybuffer, "position %4d       ", value);
  lcd.print(displaybuffer);
  while(Serial1.read()!=-1);
  }
}
```